

Mammogram Preprocessing Report

Prepared by: Allali Mohammed Abdessettar

Subject: Mammogram Image Preprocessing

Purpose: Noise removal, artifact suppression, pectoral muscle suppression, and ROI standardization

Introduction

Mammogram preprocessing is a critical step in computer-aided diagnosis (CAD) systems for breast cancer detection. The objective of preprocessing is to enhance image quality, remove irrelevant regions (background, labels, artifacts), and isolate the breast tissue for subsequent analysis. The following report explains, step by step, the preprocessing pipeline applied to a dataset of mammograms.

What Are We Doing and What Is the Main Idea?

In this work, we apply a **mammogram preprocessing pipeline** inspired by the report provided by **Dr. Benshessin Nacer**. The goal is to prepare raw mammogram images so they can be correctly analyzed by a computer-aided diagnosis (CAD) system.

Raw mammograms contain **noise, background, labels, and the pectoral muscle**, which are not useful for diagnosis and can negatively affect automated analysis. The main idea is to **remove all irrelevant information and keep only the breast tissue**, which is the medically important region.

First, we reduce noise using a median filter to improve image quality while preserving edges. Then, we separate the breast from the background and artifacts by thresholding and connected component analysis, keeping only the largest object, which corresponds to the breast. After that, we remove the pectoral muscle, which appears in MLO images and can be confused with dense tissue, by localizing it in the upper-left region and segmenting it using seeded region growing.

Finally, we crop and resize the region of interest so that all images have the same size and structure. This makes the dataset clean, consistent, and suitable for feature extraction and machine learning models.

In summary, the idea is to **simplify the mammogram image step by step**, guided by anatomical knowledge, so that later AI models focus only on relevant breast tissue. This preprocessing approach follows and adapts the methodology proposed by **Dr. Benshessin Nacer**.

Step 1: Noise Removal

Reference:

Objective:

To remove impulsive and Gaussian noise while preserving the edges, which are crucial in mammograms for identifying subtle abnormalities.

Input:

- Original grayscale mammogram
- Size: 1024×1024 pixels
- Intensity range: 0–255

Operation: Median Filtering (3×3)

Step-by-step:

1. For each pixel, consider its 3×3 neighborhood.
2. Sort the 9 pixel intensity values.
3. Replace the center pixel with the median value.
4. Slide the window over the entire image until all pixels are processed.

Rationale:

- The median filter is non-linear, preserving edges better than linear filters.
- Effective against salt-and-pepper noise, speckle noise, and small Gaussian disturbances.

Output:

- Noise-reduced mammogram with edges preserved.

Step 2: Artifact Suppression & Background Separation

Reference:

This step removes extraneous artifacts (labels, markers, wedges) and separates the breast region from the background.

2.1 Global Thresholding

Objective:

Create a binary mask separating foreground (breast + artifacts) from background.

Operation:

- Threshold $T = 18T = 18T = 18$ (normalized 0.0706)
- Pixels $\geq T \rightarrow 1$ (white), pixels $< T \rightarrow 0$ (black)

Output:

- Binary image containing the breast, artifacts, and other objects.

2.2 Connected Component Labeling

Objective:

Identify distinct objects in the binary image.

Step-by-step:

1. Scan the binary image.
2. Assign unique labels to each connected region.
3. Each object (breast, text, marker) gets a unique ID.

Output:

- Labeled binary image with all components identified.

2.3 Largest Object Selection

Objective:

Isolate the breast region.

Assumption:

The breast is the largest connected object in the image.

Step-by-step:

1. Compute area (number of pixels) for each labeled component.
2. Identify the largest component.
3. Discard all other components.

Output:

- Binary mask of the breast region only.

2.4 Mask Original Image

Objective:

Remove background and artifacts from the grayscale image.

Step-by-step:

1. Multiply the original image by the breast mask.
2. Background pixels $\rightarrow$ 0, breast pixels $\rightarrow$ unchanged.

Output:

- Clean mammogram containing only breast tissue.

Step 3: Pectoral Muscle Suppression

Reference: Section 2.1.3

Pectoral muscle suppression is essential to avoid false positives and ensure accurate breast tissue analysis.

3.1 Flip Right MLO Images

Objective:

Standardize image orientation.

Step-by-step:

1. Detect right MLO images.
2. Flip horizontally if needed.

Output:

- Pectoral muscle always appears in the upper-left corner.

3.2 Quadrant Division

Objective:

Localize the region containing the pectoral muscle.

Step-by-step:

1. Divide the 1024×1024 image into four quadrants (512×512 each).
2. Select the upper-left quadrant.

Rationale:

- Pectoral muscle is located only in the upper-left quadrant in MLO images.

3.3 Upper-Left Triangular Region Localization (LTRPM)

Objective:

Precisely localize the pectoral muscle.

Step-by-step:

1. Remove remaining background from the upper-left quadrant.
2. Divide quadrant diagonally.
3. Keep the upper-left triangle containing the pectoral muscle.

Output:

- Localized triangular region containing muscle.

3.4 Seeded Region Growing (SRG)

Objective:

Segment the pectoral muscle precisely.

Step-by-step:

1. Automatically select a seed pixel inside the triangle corresponding to the brightest region.
2. Compare neighboring pixels to the seed.
3. Add pixels with similar intensity.
4. Stop when similarity threshold is no longer satisfied.

Output:

- Binary mask of pectoral muscle only.

3.5 Remove Pectoral Muscle

Step-by-step:

1. Subtract the muscle mask from the breast image.
2. Set pectoral muscle pixels $\rightarrow 0$, preserve breast tissue.

Output:

- Breast image without pectoral muscle.

3.6 Crop Region of Interest (ROI)

Objective:

Keep only the useful breast tissue.

Step-by-step:

1. Find the bounding box around the breast region.
2. Crop using X and Y limits.

Output:

- ROI containing only breast tissue, no background or muscle.

3.7 Resize ROIs to Same Size

Objective:

Ensure uniform input for feature extraction and downstream tasks.

Step-by-step:

1. Resize all ROIs to a fixed size (e.g., 512×512).

Output:

- Standardized, clean dataset ready for analysis.

Final Result of Preprocessing

After completing the three steps:

- Noise-free images
- Artifacts removed
- Background removed
- Pectoral muscle removed
- Standardized ROIs

- Dataset ready for feature extraction and further machine learning tasks

The code:

```
import cv2
```

Imports **OpenCV**, which is used for image processing operations such as filtering, thresholding, masking, and flipping images.

```
import numpy as np
```

Imports **NumPy**, used for numerical operations, array manipulation, and image matrix handling.

```
from skimage import measure
```

Imports the **measure** module from **scikit-image**, which provides functions for connected component labeling and region analysis.

```
from skimage.segmentation import flood
```

Imports the **flood (seeded region growing)** algorithm, used to segment the pectoral muscle based on intensity similarity.

```
from skimage.transform import resize
```

Imports the resizing function to standardize ROI sizes for later processing.

```
def noise_removal_median(image):
```

Defines a function that receives a grayscale mammogram as input.

```
    Apply 3×3 median filtering to remove impulsive and Gaussian noise
    while preserving edges.
```

```
    denoised = cv2.medianBlur(image, 3)
    return denoised
```

Applies a **3×3 median filter** to the image:

Each pixel is replaced by the median value of its 3×3 neighborhood.
This removes impulsive and Gaussian noise while preserving edges

Returns the noise-reduced image.

```
def global_threshold(image, threshold=18):
```

Defines a function that converts the grayscale image into a binary image using a fixed threshold value.

```
_, binary = cv2.threshold(image, threshold, 255, cv2.THRESH_BINARY)

    return binary
```

For each pixel:

- If intensity $\geq 18 \rightarrow$ pixel becomes 255 (white)
- Else $\rightarrow$ pixel becomes 0 (black)

Returns the binary image containing breast + artifacts.

```
def connected_components(binary_image):
```

Defines a function to identify all connected regions in the binary image.

```
    labels = measure.label(binary_image, connectivity=2)

return labels
```

Labels each connected component:

connectivity=2 means 8-connected neighborhood.
Each object receives a unique integer label.

Returns the labeled image.

```
def keep_largest_component(labels):
```

Defines a function that isolates the largest connected component.

```
    regions = measure.regionprops(labels)
```

Extracts properties (area, centroid, bounding box) of each labeled object.

```
largest_region = max(regions, key=lambda r: r.area)

breast_mask = np.zeros_like(labels, dtype=np.uint8)

breast_mask[labels == largest_region.label] = 255
```

```
return breast_mask
```

Selects the component with the **maximum area**, assumed to be the breast.

Creates an empty mask with the same dimensions as the image.

Sets the pixels belonging to the breast region to white (255).

Returns the binary breast mask.

```
def apply_mask(image, mask):  
  
    masked_image = cv2.bitwise_and(image, image, mask=mask)  
  
    return masked_image
```

Defines a function to remove background and artifacts.

Keeps only pixels inside the breast mask:

Background pixels → 0

Breast pixels → unchanged

Returns the cleaned mammogram.

```
def flip_right_mlo(image, is_right_mlo):  
  
    if is_right_mlo:  
  
        return cv2.flip(image, 1)  
  
    return image
```

Defines a function to standardize orientation.

Checks if the image corresponds to a right MLO view.

Flips the image horizontally so the pectoral muscle appears in the upper-left corner.

Returns the image unchanged if it is left MLO.

```
def extract_upper_left_quadrant(image):  
  
    h, w = image.shape  
  
    return image[:h//2, :w//2]
```

Defines a function to isolate the region where the pectoral muscle is located.

Retrieves the height and width of the image.

Extracts the upper-left quadrant (512×512).

```
def upper_left_triangle(quadrant):  
  
    h, w = quadrant.shape  
  
    triangle_mask = np.zeros_like(quadrant, dtype=np.uint8)  
  
    for i in range(h):  
        for j in range(w):  
            if j <= (w - i):
```

```
triangle_mask[i, j] = 255

triangle = cv2.bitwise_and(quadrant, quadrant, mask=triangle_mask)

return triangle, triangle_mask
```

Defines a function to localize the triangular muscle region.

Gets the dimensions of the quadrant.

Creates an empty triangular mask.

Loops over every pixel.

Defines the diagonal condition to form the triangular region.

Marks triangle pixels as white.

Applies the triangle mask to the quadrant.

Returns the localized triangular region.

```
def seeded_region_growing(triangle):

    # Automatically select brightest pixel as seed

    seed_point = np.unravel_index(np.argmax(triangle), triangle.shape)

    # Normalize image for region growing

    norm_triangle = triangle / 255.0
```

```

# Region growing using flood

muscle_mask = flood(norm_triangle, seed_point, tolerance=0.15)

muscle_mask = muscle_mask.astype(np.uint8) * 255

return muscle_mask

```

Defines a function to segment the pectoral muscle.

Selects the brightest pixel automatically as the seed point.

Grows the region from the seed using intensity similarity.

Converts the mask to binary format.

Returns the pectoral muscle mask.

```

def remove_pectoral_muscle(image, muscle_mask):

    """

    Remove pectoral muscle from breast image.

    """

    muscle_mask_inv = cv2.bitwise_not(muscle_mask)

    clean_image = cv2.bitwise_and(image, image, mask=muscle_mask_inv)

    return clean_image

```

Defines a function to remove muscle pixels.

Inverts the muscle mask.

Sets muscle pixels to zero.

Returns the breast image without muscle.

```

def crop_roi(image):

    """

    Crop the bounding box around the breast region.

```

```

"""

coords = cv2.findNonZero(image)

x, y, w, h = cv2.boundingRect(coords)

roi = image[y:y+h, x:x+w]

return roi

```

Defines a function to crop the region of interest.

Finds all non-zero (breast) pixels.

Computes the bounding box.

Crops the image.

Returns the ROI.

```

def resize_roi(roi, size=(512, 512)):

    resized = resize(roi, size, preserve_range=True).astype(np.uint8)

    return resized

```

Defines a function to standardize ROI size.

Resizes the ROI without changing intensity scale.

Returns the resized ROI.

```

def preprocess_mammogram(image_path, is_right_mlo=False):

    """

    Complete mammogram preprocessing pipeline.

    """

    # Load grayscale image

    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

```

```

# STEP 1

image = noise_removal_median(image)

# STEP 2

binary = global_threshold(image)

labels = connected_components(binary)

breast_mask = keep_largest_component(labels)

breast_image = apply_mask(image, breast_mask)

# STEP 3

breast_image = flip_right_mlo(breast_image, is_right_mlo)

quadrant = extract_upper_left_quadrant(breast_image)

triangle, _ = upper_left_triangle(quadrant)

muscle_mask = seeded_region_growing(triangle)

breast_no_muscle = remove_pectoral_muscle(breast_image,
muscle_mask)

roi = crop_roi(breast_no_muscle)

final_image = resize_roi(roi)

return final_image

```

Defines the complete preprocessing pipeline.

Loads the mammogram as a grayscale image.

Applies noise removal.

Performs thresholding.

Labels connected components.

Keeps the breast region only.

Removes background and artifacts.

Extracts muscle region.

Localizes triangular muscle area.

Removes the muscle.

Crops ROI.

Returns the final preprocessed mammogram.

